1 Functions

R functions work much like SAS macros. They allow us to write code efficiently, removing the need to repeat similar bits of code when applying the same process to different sets of parameters.

By the end of this chapter you should be able to:

- Create functions and understand how we can use them to make our code more efficient.
- Understand lists at a basic level and how we can use them to output multiple objects from a function.
- Add custom error and warning messages to our functions.
- Identify some of the keys issues that we can encounter when writing our own functions.
- Add a roxygen header to a custom function and understand their uses in R package development.

1.1 Introduction to Functions

In R, there are two different types of functions:

- Built-in functions - R has many built-in functions that already exist as part of `base` R or other R packages, such as the `tidyverse`. Examples include `mean()` and `seq()` from `base` R, and `filter()`, `group_by()` and `select()` from the `tidyverse`.
- User-defined functions - We can also create and use our own functions, referred to as user-defined functions. Once created, we can use them just like any other built-in function.

1.1.1 User-Defined Functions

In order to create our own functions (called user-defined functions) we use the following basic construct:

```
function(arglist) {
  body
}
```

The word `function()` indicates to R that we want to create our own user-defined function. Next, we follow this with a list of our function's arguments, contained within parentheses `()`. Finally, we write the body of the function, which is the code that tells the function what to do, contained within curly brackets `{}`. To explain this, let's look at a simple example.

Here, we define a function named `square()`, which takes a number `x` and returns the square of the number.

```
# Define the square function
square <- function(x) {
  x^2
}
```

Breaking this down, we have started with the word `function()` to tell R that we want to create a function. Next, we have put `x` in parentheses to tell R that we want to pass one argument to the function, named `x`. Then, we write the code to define what our function actually does within curly brackets `{}`.

Finally, you may have noticed that we used the assignment operator `<-` here. This stores our new function in the environment as an object called “square”. In order to call this function, we call it like we would any other R function, by typing the name of the function, followed by the value we want to give to the function’s argument.

```
# Test the function
square(x = 3)
```

```
[1] 9
```

1.1.2 Function Arguments

When defining `square()`, we provided an argument, `x`. The name “x” is arbitrary and we can provide any valid R names as function inputs.

Valid R names consist of any combination of letters, numbers, periods `.` and underscores `_`, but can’t begin with an underscore `_` or a digit.

For `square()` we provided a single argument, but we can provide as many arguments as necessary. Here’s another function that takes two numbers and simply adds them together.

```
# Define the add function
add <- function(a, b) {
  a + b
}
# Test the function
add(a = 5, b = 7)
```

```
[1] 12
```

Note that we did not specify the type of variables that our function requires as inputs. We can therefore pass any sensible values to our function and R will attempt to add them together.

```
# Test the function
add(a = 1:5, b = 1)
```

```
[1] 2 3 4 5 6
```

Of course, the onus is on the user to provide sensible input values. For example, if we were to use a character variable as an input, the function `add()` would produce an error message.

```
# This will produce an error because we are adding a number and a string!
add(a = 5, b = "You know I'm bad, I'm bad come on, you know")
```

```
Error in a + b : non-numeric argument to binary operator
```

Just like any other functions you have used in R, the use of the argument names when calling the function is optional:

```
add(1, 2)
```

```
[1] 3
```

However, if we don't specify argument names, it's important to pay close attention to the order of the values we supply, to ensure that the values are assigned to the intended parameters.

To illustrate the importance of this, let's create a function to calculate a person's BMI.

```
BMI <- function(Weight, Height) {
  Weight / (Height^2)
}
```

Now, let's use the function to calculate the BMI of a 68kg person with a height of 1.75m.

```
BMI(68, 1.75)
```

```
[1] 22.20408
```

This call to our function seems to produce a sensible BMI for this person.

However, if we were to accidentally enter the values in the wrong order (i.e. swap around the weight and height values) our answer wouldn't be quite so sensible.

```
BMI(1.75, 68)
```

```
[1] 0.0003784602
```

In this case, it's quite easy to tell that we've done something wrong, but it might not be so easy in other cases. Therefore, we need to be careful that, when calling a function without specifying argument names, we enter the parameter values in the correct order.

1.1.3 Default Values

Additionally, we can define default values for any argument. If no value is provided to the argument, it will resolve to the default value. Alternatively, if a value is provided, the default value will be overwritten. To set

a default value for an argument, we simply use "`=`", i.e. `[arg_name] = [default_value]`.

```
# Define the add function
add <- function(a, b = 0) {
  a + b
}
```

```
# Test the function, allowing b to take the default value
add(a = 5)
```

```
[1] 5
```

```
# Test the function, overwriting b's default value
add(a = 5, b = 7)
```

```
[1] 12
```

1.2 Environment and Output

A function can only return a single object and by default, the last line of the function determines which object is returned. Upon completion of the function, all other intermediary objects are deleted (i.e. `a2` and `b2` in the example below).

```
# Define a function that squares two values and adds them together
add_squares <- function(a, b) {
  a2 <- a^2
  b2 <- b^2
  # The last line is returned
  a2 + b2
}
# Test the function
add_squares(2, 3)
```

```
[1] 13
```

```
# Intermediate objects are not returned
a2
```

```
Error: object 'a2' not found
```

However, there are some instances where we might want an object to be returned that isn't in the last line of code and therefore isn't outputted by default. For example, when using conditional "if-else" structures in functions, which will be covered in section 3.2.

To overwrite the default output object, we can use the `return()` function to specify the object to output. Once the `return()` function is called, the function exits and stops processing subsequent lines of code.

```r
# Define a function that squares two values and adds or subtracts them based on the operation sup
add_subtract_squares <- function(a, b, operation) {
  a2 <- a^2
  b2 <- b^2
  if (operation == "add") {
    return(a2 + b2)
  } else if (operation == "subtract") {
    return(a2 - b2)
  }
}
# Test the function
add_subtract_squares(2, 3, "add")
```

```
[1] 13
```

It is actually considered good practise to use the `return()` function when creating longer and more complex functions, as it makes code easier to read and understand.

1.3 Lists

As mentioned earlier, functions can only output one object. However, there is a work-around for this, which is using something called a list.

Lists are R objects which can store things like vectors and matrices as its elements. So, for example, whilst we would normally store vectors in R in their own individual objects, we can use lists to store multiple vectors in a single object.

Unlike objects like vectors, lists can contain elements of different types, such as a combination of numbers, strings and logicals. They can also contain a combination of vectors, matrices, other lists and functions.

Therefore, if we want to output multiple pieces of information from a function, we can use a list to store it all in one object and then extract specific pieces of information by accessing particular elements of the list.

1.3.1 Creating a List

A list is created using the `list()` function.

A list is made up of elements. We can give names to elements to access them.

```r
# Create a list containing strings, numbers, vectors and a logical values.
list_data <- list("Red", "Green", c(21, 32, 11), TRUE, 51.23, 119.1)
list_data
```

```
[[1]]
[1] "Red"

[[2]]
```

```
[1] "Green"

[[3]]
[1] 21 32 11

[[4]]
[1] TRUE

[[5]]
[1] 51.23

[[6]]
[1] 119.1
```

```r
# You can give names to the elements in the list
names(list_data) <- c("color1", "color2", "numbers", "a", "double1", "double2")
list_data
```

```
$color1
[1] "Red"

$color2
[1] "Green"

$numbers
[1] 21 32 11

$a
[1] TRUE

$double1
[1] 51.23

$double2
[1] 119.1
```

```r
# Alternatively
list_data <- list(
  color1 = "Red",
  color2 = "Green",
  numbers = c(21, 32, 11),
  a = TRUE,
  double1 = 51.23,
  double2 = 119.1
)
```

You can merge many lists into one list by placing all the lists inside one `list()` function.

```r
# Create two lists.
list1 <- list(1, 2, 3)
```

```
list2 <- list("Sun", "Mon", "Tue")
# Merge the two lists.
merged_list <- list(list1, list2)
merged_list
```

```
[[1]]
[[1]][[1]]
[1] 1

[[1]][[2]]
[1] 2

[[1]][[3]]
[1] 3


[[2]]
[[2]][[1]]
[1] "Sun"

[[2]][[2]]
[1] "Mon"

[[2]][[3]]
[1] "Tue"
```

```
# Accessing elements in a list of lists
merged_list[[1]]
```

```
[[1]]
[1] 1

[[2]]
[1] 2

[[3]]
[1] 3
```

1.3.2 Accessing Elements in a List

We can access elements in a list in a similar fashion to to vectors.

Integer, logical or character vectors can be used for indexing.

The words ***elements*** and ***components*** can be used interchangebly in the context of lists.

Accessing elements inside a single list using an integer:

```
list_data
```

```
$color1
[1] "Red"

$color2
[1] "Green"

$numbers
[1] 21 32 11

$a
[1] TRUE

$double1
[1] 51.23

$double2
[1] 119.1
```

```
list_data[3]
```

```
$numbers
[1] 21 32 11
```

Accessing elements inside a single list using a character:

```
list_data["double1"]
```

```
$double1
[1] 51.23
```

Using negative integers to **exclude** single list elements:

```
list_data[-2]
```

```
$color1
[1] "Red"

$numbers
[1] 21 32 11

$a
[1] TRUE

$double1
[1] 51.23

$double2
[1] 119.1
```

Accessing elements inside a single list using index vectors:

```
# Index using integer vector
list_data[1:2]
```

```
$color1
[1] "Red"

$color2
[1] "Green"
```

```
# Index using logical vector
list_data[c(TRUE, FALSE, FALSE, TRUE, FALSE, FALSE)]
```

```
$color1
[1] "Red"

$a
[1] TRUE
```

```
# Index using character vector
list_data[c("numbers", "a")]
```

```
$numbers
[1] 21 32 11

$a
[1] TRUE
```

Accessing elements in a list of lists:

```
merged_list[[1]]
```

```
[[1]]
[1] 1

[[2]]
[1] 2

[[3]]
[1] 3
```

Indexing with `[]` as shown above will give us a sub-list not the content inside the element. To retrieve the content, we need to use `[[]]`. However, this approach will allow us to access only a single element at a time.

Single `[]` returns a list. Double `[[]]` returns the content.

```
# Accessing the content inside elements
list_data[[1]]
```

```
[1] "Red"
```

```
list_data[["numbers"]]
```

```
[1] 21 32 11
```

```
list_data[["numbers"]][[1]]
```

```
[1] 21
```

```
merged_list[[2]][[2]]
```

```
[1] "Mon"
```

As an alternative to the square brackets `[[]]`, we can use the `$` operator.

They are both the same except that we can use the `$` to do partial matching on tags.

```
list_data$numbers
```

```
[1] 21 32 11
```

```
# Partial matching - There is only one element in the list who's name starts with the letter n
list_data$n
```

```
[1] 21 32 11
```

```
list_data$numbers[3]
```

```
[1] 11
```

1.4 Adding, Deleting and Modifying elements in lists

In this example we have a list containing information about 2 people, John and Sarah. The list gives us information about their names, age and languages they speak. We have this information stored in 3 different elements labeled: **name**, **age** and **speaks**. Both the **name** and **age** elements have vectors with 2 elements stored within them. However, the **speaks** element is made up of another list. This is because it is not uncommon for a person to speak more than one language, and in this case John speaks both English and French. You can also see in the second list that we have labeled the elements with the names of the individual who speaks those corresponding languages.

```
list_data2 <- list(
  "name" = c("John", "Sarah"),
  "age" = c(50, 32),
  "speaks" = list("John" = c("English", "French"), "Sarah" = c("English"))
```

```
)
list_data2
```

```
$name
[1] "John"  "Sarah"

$age
[1] 50 32

$speaks
$speaks$John
[1] "English" "French"

$speaks$Sarah
[1] "English"
```

1.4.1 Adding a new element to a list

We have recently learned some more information about John and Sarah, and we want to add this information to our `list_data2` object. We know that John is married but Sarah is not.

Since marital status is a binary piece of information, you can either be married or not, we will use the boolean type form to represent this information.

In section 2.3, we learnt how to access elements in a list using `[]` and we also learnt how to access the content of an element inside a list using `[[]]`.

We want to create a new element in `list_data2` called “married” and are then input some content inside it.

If we created a new element called **married** without any content, R would fail to create the new element. This will make more sense when we see how to delete elements.

```
# Adding a element to a list without content
list_data2[["married"]]
```

```
NULL
```

```
list_data2
```

```
$name
[1] "John"  "Sarah"

$age
[1] 50 32

$speaks
$speaks$John
[1] "English" "French"
```

```
$speaks$Sarah
[1] "English"
```

```
# Adding a new element with content
list_data2[["married"]] <- c(TRUE, FALSE)
list_data2
```

```
$name
[1] "John"  "Sarah"

$age
[1] 50 32

$speaks
$speaks$John
[1] "English" "French"

$speaks$Sarah
[1] "English"


$married
[1]  TRUE FALSE
```

1.4.2 Deleting elements from a list

We can delete a element by setting its contents to `NULL`.

```
list_data2[["age"]] <- NULL
list_data2
```

```
$name
[1] "John"  "Sarah"

$speaks
$speaks$John
[1] "English" "French"

$speaks$Sarah
[1] "English"


$married
[1]  TRUE FALSE
```

1.4.3 Modifying elements in a list

Here we want to add a new individual to the list. His name is Harry, he speaks both Spanish and French and he is unmarried. We already have named elements that can represent this information we know about

Harry. Our aim is to add new vector elements to the named elements **name**, **speaks** and **married**.

We access the element **name** using its integer index using `[[]]` to access it. We then add vector element 3 with the content **Harry** by also enclosing the number 3 also in `[[]]`.

```
list_data2[[1]][[3]] <- "Harry"
list_data2["name"]
```

```
$name
[1] "John"  "Sarah" "Harry"
```

We access the element **speaks** which is of type list using its integer vector and then add a new element named **Harry** which contains the vector elements **Spanish** and **French**.

```
list_data2[[2]][["Harry"]] <- c("Spanish", "French")
list_data2["speaks"]
```

```
$speaks
$speaks$John
[1] "English" "French"

$speaks$Sarah
[1] "English"

$speaks$Harry
[1] "Spanish" "French"
```

We access the element **married** using it's integer vector and then add a new vector element 3 with the content FALSE.

```
list_data2[[3]][[3]] <- FALSE
list_data2["married"]
```

```
$married
[1]  TRUE FALSE FALSE
```

```
list_data2
```

```
$name
[1] "John"  "Sarah" "Harry"

$speaks
$speaks$John
[1] "English" "French"

$speaks$Sarah
[1] "English"
```

```
$speaks$Harry
[1] "Spanish" "French"


$married
[1]  TRUE FALSE FALSE
```

1.4.4 Converting lists to vectors

A list can be converted to a vector so that the elements of the vector can be used for further manipulation. All the arithmetic operations on vectors can be applied after the list is converted into vectors. To do this conversion, we use the `unlist()` function. It takes the list as input and produces a vector. `unlist()` is also a `base R` function.

Arithmetic operators such as `+`, `-`, `*`, `/`, `%%`, `%/%`, `^` can be applied to vectors but cannot be applied to lists. Vectors are also useful for **storing and retrieving similar types of data**.

```
# Create two lists
list1 <- list(1:5)
list2 <- list(10:14)
# Convert the lists to vectors.
v1 <- unlist(list1)
v2 <- unlist(list2)
# Add vectors together
result <- v1 + v2
result
```

```
[1] 11 13 15 17 19
```

1.5 Returning Multiple Objects

R functions cannot return multiple objects. However, we can use lists to group multiple objects together, giving us a work-around for this problem.

Let's return to the `add_squares()` example. We could use a list in order to return the square of each input *and* their sum.

```
# Define a function that squares two values and adds them together
add_squares <- function(a, b) {
  a2 <- a^2
  b2 <- b^2
  # Return a list of results
  list(a_squared = a2, b_squared = b2, sum = a2 + b2)
}
# Test the function
add_squares(2, 3)
```

```
$a_squared
[1] 4

$b_squared
[1] 9

$sum
[1] 13
```

Another good example is R's linear modeling function, which is able to return lots of information through putting multiple objects into a list. Let's try applying this to the `vs` data set below.

```
# Load the haven package
library(haven)
# Read in vs data
vs <- read_sas("data/vs.sas7bdat")
```

	USUBJID	HEIGHT	WEIGHT
1	STD123456:000001	165	88
2	STD123456:000002	179	100
3	STD123456:000003	182	81
4	STD123456:000004	166	91
5	STD123456:000005	169	73.6
6	STD123456:000006	164	66
7	STD123456:000007	157	107
8	STD123456:000008	189	101
9	STD123456:000009	155	54.1
10	STD123456:000010	171	88

```
# Fit a linear model
my_lm <- lm(WEIGHT ~ HEIGHT, data = vs)
# Since this is a named list, what does the list contain?
names(my_lm)
```

```
[1] "coefficients"  "residuals"     "effects"       "rank"
[5] "fitted.values" "assign"        "qr"            "df.residual"
[9] "xlevels"       "call"          "terms"         "model"
```

As you can see, there are 12 objects named above, which represent different pieces of information produced from the linear model. Putting these objects into a list is what allows R to output all of this information.

We can then access specific pieces of information from this list, for example the first 5 residual values.

```
# Extract residuals
my_lm$residuals[1:5]
```

```
        1         2         3         4         5 
 6.721724  9.455875 -11.529664  9.059877 -10.325662 
```

Breaking this down, we can see that `my_lm$residuals` tells R that we want to access the vector called `residuals` from the list `my_lm`. Then, we use `[1:5]` to tell R that we specifically want elements 1 to 5 from this vector.

1.6 Errors and Warnings

Errors and warnings most often occur when functions or syntax are used in a way that it is not intended to be used. For example, as we demonstrated with our `square()` function earlier in the chapter, adding a numeric and character object produces an error.

```
# Adding a character and numeric object
"This doesn't make any sense" + 3
```

```
Error in a + b : non-numeric argument to binary operator
```

Similarly, adding two character objects also produces the same error.

```
# Adding two character objects
"Does this make any sense?" + "I don't think so."
```

```
Error in a + b : non-numeric argument to binary operator
```

In this case, the misuse of the `+` is fairly obvious, as it doesn't make any sense to arithmetically add a number and a string, or two strings. However, there are many instances where it's easy for us to misuse a function or piece of syntax without realising it. So, it's important to understand the error or warning that R produces in the console.

As the meaning of error and warning messages in R aren't always obvious, we often have to work out what R is telling us. Over time, you will learn to recognise common errors and warnings, but a quick Google search is often an effective way of finding out what a message means. Alternatively, the Statistics Globe has a quick guide on the topic, which contains information about many of the common R error and warning messages. This can be found here: Handling Errors & Warnings in R.

1.6.1 Custom Error and Warning Messages

In R, it is possible to add custom error and warning messages to your function. Before we dive into this, let's make distinctions between the three different constructs related to errors in R.

- **Errors** - This means that R cannot process the code that you have provided. So, it will halt processing of your code and (hopefully) print an error message to the console. We saw an example of this when we tried to arithmetically add strings above.
- **Warnings** - This indicates that something slightly less serious has gone wrong in your program that should be inspected. However, R is able to find an appropriate way to deal with an issue of this type (as we will see in the example below) and processing is not halted. A warning message will be printed to the console. Here's a simple example of a warning being generated, where we're trying to convert a vector of values into numeric types:

```
as.numeric(c("5", "6", "seven"))
```

```
Warning: NAs introduced by coercion

Warning: NAs introduced by coercion

[1]  5  6 NA
```

The `as.numeric()` function tries to convert each string into a number, but R can't convert "seven" to a numeric type, so a warning is generated. However, R is able to resolve this issue by producing an NA for "seven" instead of a number and therefore execution of the code doesn't need to be halted.

- **Messages** - These are simply printed to the console to provide information to the user. This has no effect on the processing or output.

Now that we can differentiate between the different types of messages, let's have a look at how we can implement these into our own user-defined functions.

Generating Errors

The first function we can use to generate error messages (and therefore stop the processing of our code) is using the `stop()` function.

```
stop("Something erroneous has occurred!")
```

```
[1] "Error: Something erroneous has occurred!"
```

However, we will generally want our user-defined function to error if a specific condition is not met. To achieve this, we can use a variant of the `stop()` function, called `stopifnot()`. This function will stop processing of the code and print an error message if any of the conditions provided are not met. If we use `stopifnot()` within a function we create, then the error message will specify the function call that generated the error and which condition in `stopifnot()` wasn't satisfied.

```
# Create a simple function to print the value of n
print_n <- function(n) {
```

```
  # Error if n is not equal to or less than 0
  stopifnot(n <= 0)
  n
}

print_n(5)
```

```
[1] "Error in print_n(5) : n <= 0 is not TRUE"
```

Generating Warnings

Similarly, we also have a function to create warning messages, called `warning()`.

```
warning("Consider yourself warned!")
```

```
Warning message:
Consider yourself warned!
```

Just like before, if we use `warning()` inside a user-defined function, R will tell us the function call which caused the warning message to be generated.

```
pizza_slices <- function(num_of_people) {
  if (num_of_people > 8) {
    warning("One pizza is not enough for more than 8 people!
            You might want to get some more pizzas!")
  } else {
    paste0("Each person gets 1/", num_of_people, " of the pizza")
  }
}

pizza_slices(4)
```

```
Each person gets 1/4 of the pizza
```

```
pizza_slices(10)
```

```
Warning message:
In pizza_slices(10) :
One pizza is not enough for more than 8 people! You might want to get some more pizzas!
```

It's useful to remember that, when using `warning()`, processing of the code will not be stopped, unlike with `stopifnot()`.

Generating Messages

Messages are much simpler than errors or warnings, as they aren't used in situation where we need to stop processing of the code or alter the output. A message can be print to the log using the `message()` function.

```
message("in a bottle.")
```

```
[1] "in a bottle."
```

1.6.2 Custom Messages in an Example

To demonstrate how we might use errors, warnings and messages, let's define a new function called `add_sqrts()`. This function will take the square root of two numbers and add them together. Notice that the combination of `round()` and `format()` functions rounds the result to one decimal place and ensures the display of that decimal place.

```
# Define a function that square-roots two values and adds them together
add_sqrts <- function(a, b) {
  a_sqrt <- sqrt(a)
  b_sqrt <- sqrt(b)

  format(round(a_sqrt + b_sqrt, 1), nsmall=1)
}

# Test the function
add_sqrts(4, 4)
```

```
[1] "4.0"
```

Let's make some additions to our `add_sqrts()` function so that it's more fool-proof. We're going to address a few issues here:

- Firstly, let's make our function error if the user provides a non-numeric argument to `add_sqrts()`. To do this, we're going to use the `stopifnot()` function, as well as `is.numeric()` to test if `a` and `b` are numeric types.

```
# Define a function that square-roots two values and adds them together
add_sqrts <- function(a, b) {

  # Error if a or b isn't a number
  stopifnot(is.numeric(a), is.numeric(b))

  a_sqrt <- sqrt(a)
  b_sqrt <- sqrt(b)

  format(round(a_sqrt + b_sqrt, 1), nsmall=1)
}
```

- Secondly, let's add a warning message if the user provides any negative numbers, using `warning()`. We can't square root a negative number, so we're going to take the absolute value of each number, using the `abs()` function. If the user does provide a negative number, our function will have to change this number to make it positive and so we want to provide a warning message informing the user that this has happened. Notice that we have used if statements to **only** print the warning message if a negative number is provided.

To produce a dynamic warning message, we have used the `paste0()` function. This will create a string with variables inside it, that will resolve to the values the user provides to `add_sqrts()`. This is similar to the cat functions in SAS.

In line with the `tidyverse` framework, can you find a function in the `stringr` package that would do the same job as `paste0()`?

```
# Define a function that square-roots two values and adds them together
add_sqrts <- function(a, b) {

  # Error if a or b isn't a number
  stopifnot(is.numeric(a), is.numeric(b))

  # Warning if a is less than 0
  if (a < 0) {
    warning(paste0("You can't square root ", a, " (argument a) as it's a negative number! ", a, "
  }
  # Warning if b is less than 0
  if (b < 0) {
    warning(paste0("You can't square root ", b, " (argument b) as it's a negative number! ", b, "
  }

  a <- abs(a)
  b <- abs(b)

  a_sqrt <- sqrt(a)
  b_sqrt <- sqrt(b)

  format(round(a_sqrt + b_sqrt, 1), nsmall=1)
}
```

- Finally, let's make sure the user knows that the result will be rounded to one decimal place. We're going to do this using the `message()` function.

```
# Define a function that square-roots two values and adds them together
add_sqrts <- function(a, b) {

  # Error if a or b isn't a number
  stopifnot(is.numeric(a), is.numeric(b))

  # Warning if a is less than 0
  if (a < 0) {
    warning(paste0("You can't square root ", a, " (argument a) as it's a negative number! ", a, "
  }
  # Warning if b is less than 0
  if (b < 0) {
    warning(paste0("You can't square root ", b, " (argument b) as it's a negative number! ", b, "
  }
```

```
  a <- abs(a)
  b <- abs(b)

  # Let the user know that this function rounds the result to 1 decimal place
  message("This function rounds the result to 1 decimal place.")

  a_sqrt <- sqrt(a)
  b_sqrt <- sqrt(b)

  format(round(a_sqrt + b_sqrt, 1), nsmall=1)
}
```

Let's now test each error, warning and message element of our new function!

Error

```
# Provide a string as an argument
add_sqrts(4, "Hi!")
```

```
[1] "Error in add_sqrts(4, 'Hi!') : is.numeric(b) is not TRUE"
```

Warning

```
# Provide a string as an argument
add_sqrts(4, -4)
```

```
This function rounds the result to 1 decimal place.
[1] 4
Warning message:
In add_sqrts(4, -4) :
  You can't square root -4 (argument b) as it's a negative number!
-4 will default to it's absolute value, 4.
```

Notice that our warning message says what the value of argument b is, rather than just saying "argument b". As a reminder, we've been able to make this warning message dynamic in this way by using the `paste0()` function.

Normal processing (with message)

```
# Provide a string as an argument
add_sqrts(4.5, 4)
```

```
This function rounds the result to 1 decimal place.

[1] "4.1"
```

As you can see, `add_sqrts()` now has some validation steps in place, to ensure the user is using the function properly!

1.7 EXERCISE

A subject's USUBJID is made up of their STUDYID and SUBJID of the first study they participated in, separated by a period `.`. Let's create a function to extract some information from a given USUBJID. For this exercise, you can assume that all STUDYIDs are 9 characters long and start with the prefix "STD", and all SUBJIDs are 6 characters long. You should test your function on the USUBJID "STD123456.000703" after you've finished each question.

1. Create a function which returns the STUDYID for the study which our subject is on (the first 9 characters of the USUBJID).

Hint: Try using the `str_sub()` function

2. Update the function that you just created, so that it also returns the SUBJID (the last 6 characters of the USUBJID).

Hint: Try using a `list()`, so that your function can return both pieces of information.

3. If we were to refer casually to this subject, we would probably say "subject 703" rather than "subject 000703". Update your function so that it also returns the SUBJID without the leading 0's.

Hint: The REGEX expression "^0+" represents "one of more 0's at the start of a string".

4. Let's add some extra customisations to your function, so that appropriate errors and warnings come up if the user enters an invalid USUBJID.

a. Add some code to your function so that it errors if the USUBJID provided isn't 16 characters in length.

b. Update your function so that it provides a warning message to the user if the STUDYID doesn't begin with the prefix "STD".

1.8 Functions using `dplyr`

`dplyr` functions make interactive data exploration fast and fluid, but they add some new challenges when trying to use them within user-defined functions.

Most `dplyr` functions use something called tidy evaluation, which is a special, non-standard method of evaluation used throughout the `tidyverse`. Simply put, when creating a user-defined function which

contains `tidyverse` functions, we may need to supply arguments to the `tidyverse` functions slightly differently than normal. This is best explained via an example.

To find out more about tidy evaluation, try reading this article on Programming with dplyr.

Let's say we want to create a function which finds the average age of subjects based on different by groups. For example, we might want to find the average age of subjects in each sex. We're going to use some demographics data for this, which you can see below.

```
Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

	USUBJID	AGE	AGEU	SEX	RACE	ETHNIC
1	01-701-1015	63	YEARS	F	WHITE	HISPANIC OR LATINO
2	01-701-1023	64	YEARS	M	WHITE	HISPANIC OR LATINO
3	01-701-1028	71	YEARS	M	WHITE	NOT HISPANIC OR LATINO
4	01-701-1033	74	YEARS	M	WHITE	NOT HISPANIC OR LATINO
5	01-701-1034	77	YEARS	F	WHITE	NOT HISPANIC OR LATINO
6	01-701-1047	85	YEARS	F	WHITE	NOT HISPANIC OR LATINO
7	01-701-1057	59	YEARS	F	WHITE	HISPANIC OR LATINO
8	01-701-1097	68	YEARS	M	WHITE	NOT HISPANIC OR LATINO
9	01-701-1111	81	YEARS	F	WHITE	NOT HISPANIC OR LATINO
10	01-701-1115	84	YEARS	M	WHITE	NOT HISPANIC OR LATINO

Based off our current knowledge of functions, we might write the following function to achieve our goal.

```r
# Load the relevant libraries
library(haven)
library(DT)
# Use this code to load in dm and replicate the data above
dm <- read_xpt("data/dm.xpt") %>%
  select(USUBJID, AGE, AGEU, SEX, RACE, ETHNIC)
```

```r
age_summary <- function(data, group_var) {
  data %>%
    group_by(group_var) %>%
    summarise(avg_age = mean(AGE))
}
```

However, when we try to use this function to find the average age of subjects by sex, we get the following error.

```r
age_summary(dm, SEX)
```

```
Error: Must group by variables found in `.data`.
* Column `group_var` is not found.
```

This is because `dplyr` functions, such as `group_by()`, use tidy evaluation. As a result, R thinks that we are trying to refer to a variable in the data called “group_var”. However, we actually want R to resolve “group_var” to the variable that the user supplies, in this case `SEX`.

To resolve this, we must surround the input to the `group_by()` function in double curly brackets `{{}}`, as seen below.

```r
age_summary <- function(data, group_var) {
  data %>%
    group_by({{ group_var }}) %>%
    summarise(avg_age = mean(AGE))
}
```

When we call the function this time, R will recognise that we are trying to supply a variable name to the `group_by()` function and will react accordingly.

```r
age_summary(dm, SEX)
```

```
# A tibble: 2 × 2
  SEX   avg_age
  <chr>   <dbl>
1 F        75.2
2 M        75.0
```

Let’s test the function again, but this time grouping by race.

```r
age_summary(dm, RACE)
```

```
# A tibble: 4 × 2
  RACE                             avg_age
  <chr>                              <dbl>
1 AMERICAN INDIAN OR ALASKA NATIVE    61.5
2 ASIAN                               81
```

```
3 BLACK OR AFRICAN AMERICAN          72.4
4 WHITE                              75.4
```

As you can see, we have to approach the creation of user-defined functions which use `dplyr` functions slightly differently, due to the different evaluation process behind most `tidyverse` functions. However, with the addition of some new notation (the double curly brackets `{{}}`), we can get around this problem fairly easily and start creating our own functions using `dplyr`!